Addendum: Engineering Environment Blueprint for Claude Code

> How to set up your physical workspace, terminal, parallelization strategy, memory system, and multi-agent coordination — drawn from the practices of Boris Cherny (Claude Code creator), the Anthropic Claude Code team, Andrej Karpathy, and leading practitioners.

---

1. The Command Center: Terminal, Tabs, and Display

Terminal Choice

The Claude Code team at Anthropic has converged on **Ghostty** as the preferred terminal emulator — multiple team members independently chose it for its fast rendering, proper 24-bit color, and clean unicode support. **iTerm2** remains the most popular alternative on macOS, especially for its system notification support (critical when juggling multiple agents).

The terminal is the interface. Unlike IDE-based AI tools, Claude Code is a CLI agent. Your terminal *is* your IDE for directing agents. Invest in it accordingly.

The Tab-Per-Agent Layout

Boris Cherny runs 5 Claude Code instances simultaneously in numbered terminal tabs (1–5). He uses iTerm2 system notifications to know when any instance needs input. This is the foundational pattern: **one tab per agent, numbered for fast switching**.

The team extends this with color-coding and naming — one tab per task or worktree. Some use tmux for this, others native terminal tabs. The mechanism matters less than the discipline: every agent gets its own clearly-labeled home.

Configure your status line to always show context usage and current git branch:

```
/statusline
```

This gives you at-a-glance awareness of each agent's state without switching into it.

Voice Dictation for Prompts

This was one of the less obvious but high-impact tips from the Claude Code team: **use voice dictation for prompts**. On macOS, press `fn` twice to activate dictation. You speak roughly 3x

faster than you type, and — critically — your prompts become significantly more detailed and nuanced when spoken aloud. Typing encourages brevity. Speaking encourages the kind of rich context that produces better agent output.

This is especially valuable for Plan Mode prompts, where the quality of your specification directly determines whether Claude can one-shot the implementation.

---

2. Parallelization: The Single Biggest Productivity Unlock

The unanimous #1 tip from the Claude Code team is running multiple Claude sessions in parallel. Not one session doing sequential work — multiple isolated sessions working on separate tasks concurrently. This is the shift from "pair programmer" to "fleet commander," as Karpathy described it.

The Three Parallelization Surfaces

Terminal sessions (local). Boris runs 5 Claude Code instances in his terminal. Each runs in its own isolated workspace (see Section 3 below). System notifications tell him when any session needs input. This is the primary workhorse.

Web sessions (claude.ai/code). Boris also runs 5–10 additional sessions in the browser, in parallel with local sessions. He uses `&` to background a terminal session to the web, and `--teleport` to move sessions back and forth between local and web. Web sessions are ideal for tasks that don't need local file access or can run longer without supervision.

Mobile sessions. Several team members start lightweight sessions from their phone (Claude iOS app) in the morning — kicking off research, analysis, or planning tasks — then check in on them later from desktop. This turns dead time (commuting, waiting) into productive agent time.

How Many Is Too Many?

Boris operates at 10–15 concurrent sessions. But he also notes that 10–20% of sessions get abandoned due to unexpected scenarios. The practical ceiling for most engineers is **3–5 terminal sessions** plus a few web sessions. The real bottleneck isn't agent compute — it's your review bandwidth. Two well-reviewed implementations are worth more than five half-reviewed ones.

Start with 2 parallel agents. Learn the workflow. Scale up only when you can review faster than agents produce.

---

3. Workspace Isolation: Git Worktrees vs. Checkouts

Parallel agents require isolated file state. Two agents editing the same file in the same directory will produce conflicts and corruption. There are two approaches, and the Claude Code team is split on which is better.

Git Worktrees (Team Preference)

Most of the Claude Code team uses git worktrees. A worktree is a linked working directory that shares the same `.git` data as your main checkout but has its own independent file state:

```
# Create worktrees for parallel work
git worktree add ../project-feature-auth -b feature-auth
git worktree add ../project-fix-payments -b fix-payments
git worktree add ../project-refactor-api -b refactor-api
```

Each worktree gets its own branch, its own files, and its own Claude session. They share git history, so merging is straightforward. The team liked this enough that Anthropic engineer Anthony Morris built native worktree support into Claude Code.

Shell aliases for fast switching are common on the team:

```
alias za="cd ~/project-worktree-a && claude"
alias zb="cd ~/project-worktree-b && claude"
alias zc="cd ~/project-worktree-c && claude"
```

One keystroke to jump to any agent's workspace.

Separate Git Checkouts (Boris's Personal Preference)

Boris himself uses completely separate git checkouts rather than worktrees — 5 independent clones of the repo. This provides even stronger isolation (no shared `.git` directory) at the cost of more disk space and slightly more complex merging.

Worktree Gotchas

Worktrees are not without friction:

- **Shared databases and Docker.** Worktrees isolate source code, not runtime state. Two agents running migrations or modifying the same database will conflict. Use per-worktree database instances or Docker volume names for true isolation.
- **Disk space.** Each worktree duplicates your source files. With large codebases and `node_modules`, this adds up fast.

- **IDE support.** JetBrains has no native worktree UI. VS Code added support in mid-2025. Claude Code's `/ide` command can struggle with worktree paths.
- **Environment files.** `.env` files, `node_modules`, build caches — these need to be copied or symlinked per worktree. Automate this in a setup script.

A Minimal Worktree Bootstrap Script

```
#!/bin/bash
# spawn-agent.sh — Create worktree + launch Claude in a tmux session
BRANCH=$1
PROMPT=$2
PROJECT_ROOT=$(git rev-parse --show-toplevel)
PROJECT_NAME=$(basename "$PROJECT_ROOT")
WORKTREE_DIR="${PROJECT_ROOT}-${BRANCH}"

git worktree add "$WORKTREE_DIR" -b "$BRANCH" 2>/dev/null || \
  git worktree add "$WORKTREE_DIR" "$BRANCH"

cd "$WORKTREE_DIR"
cp "${PROJECT_ROOT}/.env" .env 2>/dev/null
ln -sf "${PROJECT_ROOT}/node_modules" node_modules 2>/dev/null

tmux new-session -d -s "claude-${BRANCH}" \
  "cd $WORKTREE_DIR && claude '$PROMPT'"

echo "Agent 'claude-${BRANCH}' launched in $WORKTREE_DIR"
```

---

4. tmux: The Session Persistence Layer

Why tmux Matters

tmux is not strictly required, but it solves three problems that become critical at scale:

1. **Session persistence.** tmux sessions survive SSH disconnects, terminal closures, and laptop suspends. Your agents keep working even when you're not watching. This is essential for long-running tasks and remote development.
2. **Detach and reattach.** Start an agent, detach (`Ctrl+b d`), do other work, reattach later to check results. This decouples agent execution from your attention.
3. **Multi-pane visibility.** Split your terminal to see multiple agents simultaneously without switching tabs.

tmux vs. Native Tabs

The Claude Code team is split. Some use tmux for everything. Others use native terminal tabs (iTerm2, Ghostty) and reserve tmux for specific use cases like persistent background agents or remote sessions. Boris himself uses numbered iTerm2 tabs, not tmux.

Use tmux when: you need persistence (remote dev, long tasks), you want to detach from agents and return later, or you're automating multi-agent spawning with scripts.

Use native tabs when: you prefer visual tab switching, you want system notifications per tab (tmux doesn't support this natively), or you want the simplicity of your terminal's built-in tab management.

Claude Code Agent Teams (Experimental)

Claude Code has a built-in experimental feature for coordinating multiple instances within tmux. When enabled, one session acts as team lead, assigning tasks and synthesizing results. Teammates work independently, each in their own context window, and can communicate with each other directly:

```
# Enable in settings
CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
```

Split panes require tmux or iTerm2. This feature has known limitations — no session resumption for in-process teammates, and task status can lag — but it represents the direction Claude Code is headed for built-in orchestration.

Third-Party Orchestration Tools

The ecosystem has produced several tools for managing parallel Claude sessions:

- **claude-squad** — Uses tmux for isolated terminal sessions and git worktrees per agent. Provides a TUI dashboard to monitor all agents.
- **workmux** — Tightly couples git worktrees with tmux window management. Auto-creates worktrees, configures panes, and shows agent status icons (🤖 working, 💬 needs input, ✅ done).
- **ittybitty** — Full hierarchical agent orchestration where Manager agents can spawn Worker agents, all in tmux with git worktree isolation. Agents can use CLI commands to communicate. Includes a TUI dashboard.

These tools are maturing rapidly but remain somewhat fragile. Evaluate whether their complexity is worth the management overhead for your use case.

---

5. Memory: Persisting Knowledge Across Sessions

Claude Code is stateless — every session starts from zero. Memory is how you bridge the gap. The memory system has four layers, from most persistent to most ephemeral.

Layer 1: CLAUDE.md (You Write, Claude Reads)

The team-shared project memory. Loaded every session. Checked into git. Boris's team at Anthropic keeps theirs at roughly 2,500 tokens. The entire team contributes to it multiple times a week.

The critical practice: **after every PR review where Claude made a mistake, tag `@.claude` on the PR to update CLAUDE.md.** Every mistake becomes a rule. Over time, the error rate measurably drops.

One engineer on the team takes this further: maintaining a `notes/` directory per task, updated after every PR, with CLAUDE.md pointing at it.

Layer 2: Auto Memory / MEMORY.md (Claude Writes for Itself)

Located at `~/.claude/projects/<project>/memory/MEMORY.md`. Unlike CLAUDE.md which you write, auto memory contains notes Claude writes for itself — patterns discovered, corrections logged, project facts learned. The first 200 lines are loaded into every session's system prompt.

Use the `[LEARN:tag]` format for corrections you don't want repeated:

```
## Corrections Log
- [LEARN:api] Stripe webhook signatures use raw body, not parsed JSON
- [LEARN:auth] JWT refresh tokens must be rotated on each use
- [LEARN:deploy] Staging uses port 3001, not 3000
```

These tags are searchable and persist across sessions. When Claude encounters a similar situation, it checks memory first.

Layer 3: Session Memory (Automatic Background System)

A newer feature (rolling out as of early 2026) where Claude automatically extracts summaries from each session and saves structured markdown to `~/.claude/projects/<project>/<session-id>/session-memory/summary.md`. When you start a new session, relevant memories from prior sessions are recalled automatically.

You'll see "Recalled X memories" at session start and "Wrote X memories" during your session. This works best when you state your intent early ("I'm building the payment integration using

Stripe") and summarize decisions explicitly ("We decided on webhook-based sync instead of polling").

Layer 4: External Files (The Nuclear Option for Long Tasks)

For complex, multi-session work, the most reliable memory is writing state to disk yourself. This is the "write-to-file" pattern:

```
Before ending this session, write the following to PROGRESS.md:
1. What we accomplished
2. What decisions were made and why
3. What remains to be done
4. Any gotchas or context the next session needs
```

Common file patterns:

File	Purpose
`PROGRESS.md`	What's done, what's pending, where we left off
`DECISIONS.md`	Architectural decisions and their rationale
`PLAN.md` / `SPEC.md`	The implementation plan or feature spec
`SCRATCHPAD.md`	Claude's working notes, thinking trail, file edit log

A fresh session reads these files and resumes with full signal quality — no "summary of a summary" degradation from compaction.

The Scratchpad Pattern

Multiple practitioners independently converge on telling Claude to maintain a scratchpad file during complex work:

```
Keep a running log in .claude/SCRATCHPAD.md as you work. Record:
- Files examined and key findings
- Decisions made and alternatives considered
- Changes made and why
- Open questions or concerns
Update it after each significant step.
```

This gives Claude an external working memory that survives compaction *and* session boundaries. It also gives you an audit trail of what Claude was thinking.

---

6. Plans: How to Create and Execute Them

The Plan Mode Discipline

Boris's workflow: most sessions start in Plan Mode (`Shift+Tab` twice). He iterates with Claude until the plan is solid, then switches to auto-accept edits mode for execution. Claude usually one-shots the implementation from a good plan.

The Claude Code team extends this with a powerful pattern: **one engineer has Claude write the plan, then spins up a second Claude instance to review the plan as a "staff engineer."** This two-Claude review catches architectural issues before any code is written.

Plans as Files, Not Conversation

Plans should always be written to files, not kept in conversation history:

```
Create a detailed implementation plan and write it to PLAN.md. Include:
1. Architecture overview (what changes, what stays)
2. File-by-file change list with descriptions
3. Dependency order (what must be done first)
4. Testing strategy
5. Risks and mitigations
```

This plan survives session boundaries. Multiple agents can read it. You can review and edit it in your IDE before giving Claude the go-ahead to execute.

The Spec-Then-Execute Across Sessions

The most robust pattern for large features:

1. **Session 1 (Plan):** Research, interview, write `SPEC.md`
2. **You:** Review spec in your editor. Refine. Approve.
3. **Session 2 (Execute):** "Implement the spec in SPEC.md step by step. Update PROGRESS.md as you complete each section."

Each session starts with clean context. The spec file carries all the intent and constraints. This is Karpathy's "imperative to declarative" shift made concrete — you declare *what* you want in a file, and the agent figures out how to implement it.

TodoWrite for Tracking Within a Session

For tasks that fit within a single session, instruct Claude to use structured TODO tracking:

```
Before writing any code, create a detailed TODO plan with at least
10 items covering all changes needed. Mark each with priority.
Complete items one at a time, running tests after each change.
Mark items complete as you go.
```

The TODO state is injected as a system message after each tool use, preventing goal drift during long execution loops.

---

7. Multi-Agent Coordination: Shared Plans and Progress

When multiple agents work in parallel, they need a coordination mechanism. They can't talk to each other directly (except in the experimental Agent Teams feature). The coordination layer is **the filesystem.**

The Shared Plan Pattern

For a large feature broken into parallel streams:

```
project/
├── PLAN.md                    ← Master plan (read by all agents)
├── PROGRESS.md                ← Overall status (updated by each agent)
├── streams/
│   ├── auth/
│   │   ├── SPEC.md            ← Auth stream spec
│   │   ├── STATUS.md          ← "In progress: JWT validation"
│   │   └── NOTES.md           ← Auth agent's working notes
│   ├── payments/
│   │   ├── SPEC.md
│   │   ├── STATUS.md
│   │   └── NOTES.md
│   └── frontend/
│       ├── SPEC.md
│       ├── STATUS.md
│       └── NOTES.md
```

Each agent works in its own worktree but reads the shared plan. Each updates its own `STATUS.md`. A coordinating session (or you) periodically checks status files and redirects agents as needed.

The Orchestrator Pattern

For more structured coordination, designate one session as the orchestrator:

```
You are the orchestrator for a multi-stream implementation.
Read PLAN.md for the master plan.
Read streams/*/STATUS.md for each stream's current state.
Your job:
1. Check which streams are blocked or complete
2. Update PROGRESS.md with overall status
3. Identify integration points where streams need to sync
4. Flag any conflicts between streams' approaches
Do this every time I ask "status update".
```

The orchestrator doesn't write code. It reads, coordinates, and flags issues for your attention. This mimics how a tech lead operates — maintaining the big picture while individual contributors focus on their streams.

File-Based Inter-Agent Communication

When Agent A produces something Agent B needs:

```
# In Agent A's prompt:
When you finish the database schema, write the final schema
to shared/schema.md so the API agent can read it.

# In Agent B's prompt:
Before implementing API routes, read shared/schema.md for
the current database schema. If it doesn't exist yet, tell me
the auth stream hasn't completed the schema yet.
```

This is crude but effective. The filesystem is the message bus. Status files are the coordination protocol.

Avoiding Conflicts

The golden rule: **two agents should never edit the same file.** Break work so each agent owns a different set of files. If two streams must eventually touch the same file, have one complete first and the other start from its output.

For shared resources beyond files (databases, Docker, package.json):

- Use per-worktree database instances
- Coordinate package installations through the orchestrator
- Lock shared config files to one agent at a time

---

8. The Daily Operating Rhythm

Putting it all together, here is the daily rhythm that emerges from studying how practitioners at this level work:

Morning Setup (~5 minutes)

1. Open terminal. Check existing tmux sessions or worktrees from yesterday.
2. Review `PROGRESS.md` and `STATUS.md` files from any overnight or web sessions.
3. Start 1–2 mobile sessions for background research or planning tasks.
4. Plan the day's parallel streams. What can be done independently?

Working Session

1. **Spawn worktrees** for each independent task (2–5).
2. **Plan Mode first** in each session. Iterate until the plan is right.
3. **Switch to auto-accept** once the plan is solid. Let Claude execute.
4. **Monitor via notifications.** Don't context-switch to agents proactively — let them come to you.
5. **Review output** as agents complete. Read diffs. Run tests. Check scope.
6. **Use `/compact`** at logical breakpoints. **Use `/clear`** between unrelated tasks.

Between Tasks

- Update `CLAUDE.md` with any lessons learned.
- Merge completed worktrees. Clean up with `git worktree remove`.
- Write progress to files if work will span another session.

End of Day

- Run `/compact` or write state to `PROGRESS.md` for any in-flight work.
- Tag `@.claude` on any PRs with learnings for `CLAUDE.md`.
- Abandon any stuck sessions (Boris reports 10–20% abandonment rate — this is normal).

---

9. Karpathy's Key Observations for Environment Design

Andrej Karpathy's intensive Claude coding experience (his workflow flipped from 80% manual to 80% agent in a matter of weeks around December 2025) yields several insights that should shape your environment:

The IDE is still essential — for reading, not writing. Karpathy's setup is Claude conversation windows on the left, IDE on the right for review and manual edits. The agent writes. You read, verify, and steer. Your environment should optimize for fast switching between "directing" (the terminal) and "reviewing" (the IDE or `git diff`).

Declare, don't instruct. Instead of telling Claude step-by-step how to do something, define success criteria and let it loop until they're met. This means your plans and specs should describe *what done looks like*, not *how to get there*. Build your templates accordingly.

Watch them like a hawk. For any important code, Karpathy emphasizes active review. Your environment should make it effortless to see what each agent changed — `git diff`, test output, and status files should be one keystroke away.

Expect atrophy. Your manual coding skills will rust. Combat this by using Claude as a teacher: have it explain what it wrote, generate learning materials about unfamiliar code, and ask for ASCII architecture diagrams. One team member builds spaced-repetition learning skills — Claude quizzes you on your own codebase.

---

10. Reference: Tool and Configuration Summary

Boris Cherny's Setup

Component	Choice
Terminal	iTerm2 (with system notifications)
Tabs	5, numbered 1–5
Isolation	Separate git checkouts (not worktrees)
Model	Opus 4.5 with thinking, for everything
Web sessions	5–10 on claude.ai/code
Mobile	Claude iOS, sessions started in morning
Permissions	`/permissions` with pre-allowed safe commands
Formatting	PostToolUse hook (`bun run format
Slash commands	`/commit-push-pr` (dozens of times daily)
Subagents	`code-simplifier`, `verify-app`
CLAUDE.md	~2,500 tokens, whole team contributes
MCP	Slack, BigQuery (bq CLI), Sentry

Claude Code Team Common Setup

Component	Choice
Terminal	Ghostty (multiple team members)
Isolation	3–5 git worktrees with shell aliases (za, zb, zc)
Status	`/statusline` showing context + git branch
Prompting	Voice dictation (fn×2 on macOS)
Planning	Plan Mode → iterate → auto-accept
CLAUDE.md	Evolved continuously; Claude writes its own rules
Skills	`/techdebt`, context-dump (Slack+GDrive+GitHub), analytics
Review	Two-Claude pattern (one writes plan, one reviews as staff eng)

Karpathy's Setup

Component	Choice
Layout	Claude windows (left), IDE (right)
Workflow	80% agent, 20% manual edits and touchups
Primary tools	Claude Code + Codex (for review)
Approach	Declarative specs with success criteria
Review	Active human review of all important code

Essential Shell Configuration

```
# Worktree management
alias worktrees="git worktree list"
alias wt-add="git worktree add"
alias wt-rm="git worktree remove"

# Quick agent spawning
spawn() {
  local branch=$1 prompt=$2
  git worktree add "../$(basename $(pwd))-${branch}" -b "$branch"
2>/dev/null
  cd "../$(basename $(pwd))-${branch}"
  claude "$prompt"
}
```

```
# Status check across all worktrees
agent-status() {
  for dir in ../$(basename $(pwd))-*/; do
    echo "=== $(basename $dir) ==="
    cat "$dir/STATUS.md" 2>/dev/null || echo "(no status file)"
    echo
  done
}
```

Directory Structure Template

```
~/.claude/
├── CLAUDE.md                   # Personal global preferences
├── settings.json               # Global permissions and hooks
├── skills/
│   ├── commit-push-pr/
│   ├── techdebt/
│   └── code-simplifier/
└── projects/
    └── <project>/
        └── memory/
            └── MEMORY.md       # Auto-memory per project

<project>/
├── CLAUDE.md                   # Team project rules (git-tracked)
├── CLAUDE.local.md             # Personal project prefs (git-ignored)
├── .claude/
│   ├── settings.json           # Team permissions + hooks (git-tracked)
│   ├── settings.local.json     # Personal settings (git-ignored)
│   ├── commands/               # Shared slash commands (git-tracked)
│   ├── agents/                 # Custom subagent definitions
│   └── skills/                 # On-demand knowledge files
├── .mcp.json                   # MCP server config (git-tracked)
├── PLAN.md                     # Current implementation plan
├── PROGRESS.md                 # Cross-session progress state
├── DECISIONS.md                # Architectural decision log
└── SCRATCHPAD.md               # Agent working notes
```

Closing: Start Simple, Scale Deliberately

Boris's most important meta-lesson is that his setup is "surprisingly vanilla." The power doesn't come from exotic tooling. It comes from disciplined application of fundamentals: plan before you

build, verify everything, write state to files, and let agents work in parallel without stepping on each other.

Start with:

1. One extra worktree and one extra Claude session
2. Plan Mode for every non-trivial task
3. A `CLAUDE.md` that you update after every mistake
4. A `PROGRESS.md` for anything that spans sessions

Scale up only when your review bandwidth allows it. The agents are fast. Your judgment is the bottleneck. Build the environment around optimizing *your* ability to direct, review, and decide — not around maximizing agent throughput.